BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Trương Thị Huyền Trang
MSSV / Lớp	3123410385

Mục tiêu học tập

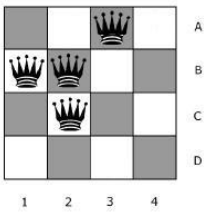
- Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).
- Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.
- Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.
- Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.
- Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Trạng thái ban đầu:

Q1: A3

Q2: B1

Q3: B2

Q4: C2

Thuật toán Hill Climbing tìm cách giảm số cặp quân hậu đang tấn công nhau bằng cách di chuyển một quân hậu hợp lệ.

Ban đầu có 4 cặp xung đột.

Sau khi xét các nước đi hợp lệ, nước đi tốt nhất là:

- Di chuyển quân hậu tại B2 đến D4

- Các quân hậu khác giữ nguyên
- Số cặp xung đột giảm từ 4 xuống 1

Quân hậu	Ban đầu	Sau 1 vòng
Q1	A3	A3
Q2	B1	B1
Q3	B2	D4
Q4	C2	C2

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng

✔ B) Cực trị cục bộ (local optimum) => Dừng vì không có nước đi đơn lẻ nào làm giảm số xung đột xuống 1

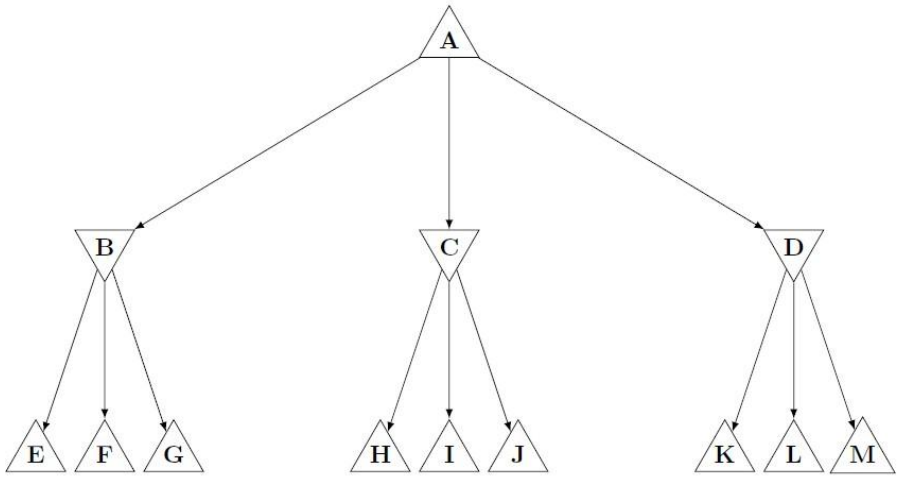
☐ C) Cực trị toàn cục (global optimum)

✔ D) Vai bằng (shoulder) => Trạng thái nằm trên một vùng bằng phẳng; có thể đi ngang qua trạng thái cùng giá trị rồi tiếp tục cải thiện, nhưng thuật toán trong đề không cho phép di chuyển ngang.

☐ E) Không thuộc các trường hợp trên

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu. a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D

☐ C → D → B

✔ D → C → B => ưu tiên giá trị đánh giá lớn hơn.

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Tại D:

K = 23; L=20; M=26 => D = min(23,20, 26) = 20

Tại gốc A: $\alpha = 20$

Xét C (min):

H = 17

Vì $\beta_C = 17 < \alpha_A = 20$ => Cắt tỉa **I và J**

Xét B(min):

E = 5

Vì $\beta_B = 5 < \alpha_A = 20$ => Cắt tỉa **F và G**

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

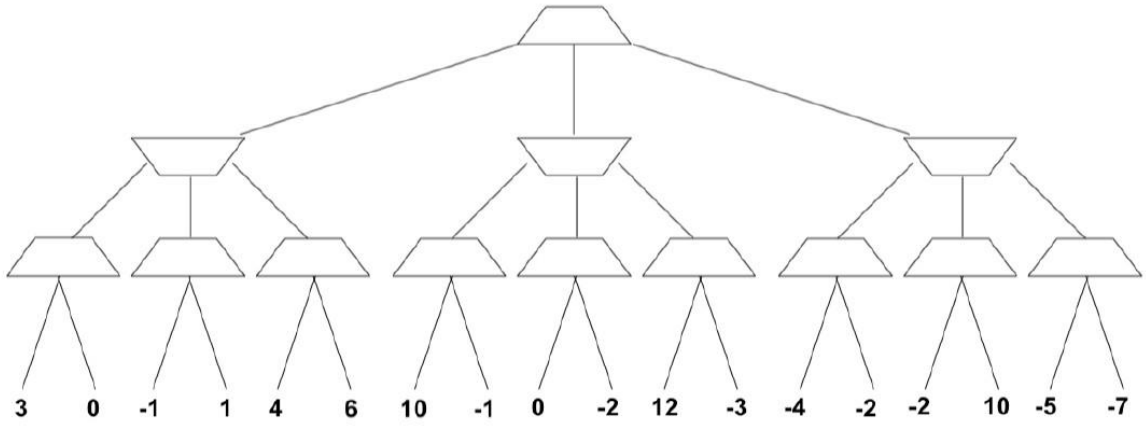
☐ Có

✔ Không

Alpha-Beta Prunning cho kết quả giống Minimax khi tìm kiếm cùng độ sâu và dùng cùng giá trị lá. Tuy nhiên, khi Iterative Deepening dừng ở độ sâu giới hạn, thuật toán phải sử dụng hàm đánh giá heuristic cho các trạng thái chưa kết thúc. Vì vậy, nước đi được chọn có thể khác với Minimax duyệt toàn bộ cây.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



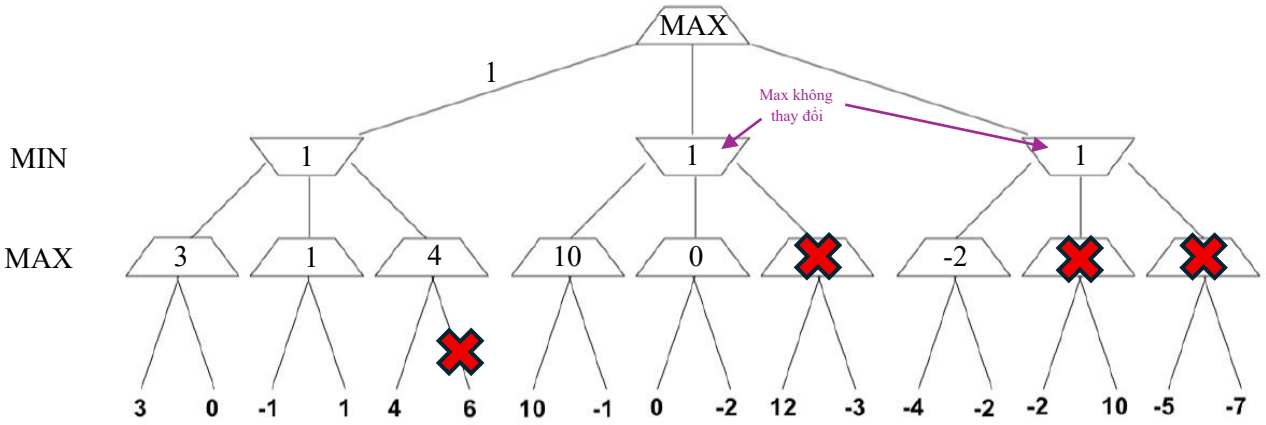
Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

- Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.
- Ghi giá trị được trả về tại mỗi nút trung gian.
- Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.
- Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

Nhánh	Giá trị các nút MAX	Giá trị MIN
Trái	max(3,0)=3; max(-1,1)=1; max(4,6)=6	min(3,1,6)=1
Giữa	max(10,-1)=10; max(0,-2)=0; max(12,-3)=12	min(10,0,12)=0
Phải	max(-4,-2)=-2; max(-2,10)=10; max(-5,-7)=-5	min(-2,10,-5)=-5

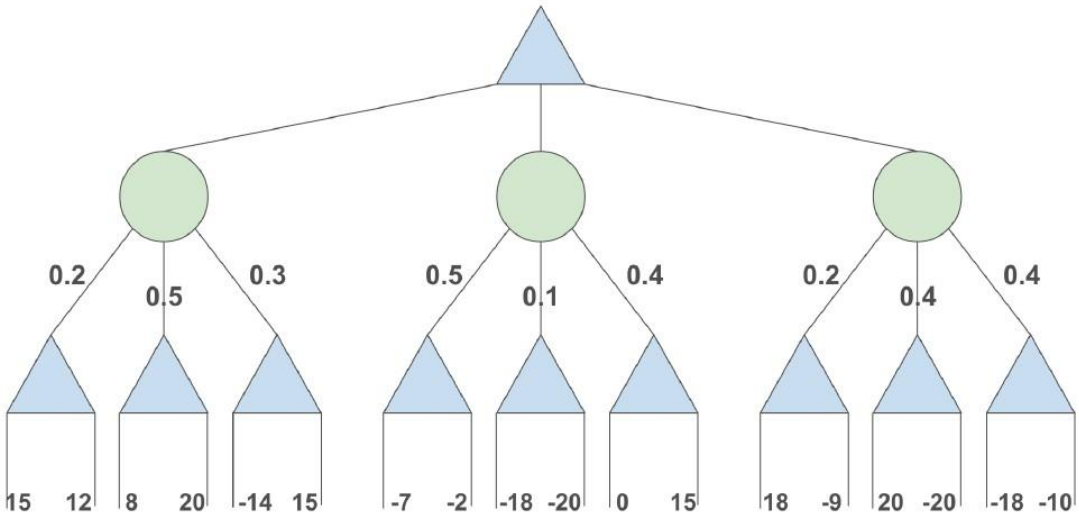
Giá trị gốc: Max(1,0,-5) = 1



Vậy, A chọn nhánh bên trái => Minimax(A) = 1

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Nhánh trái:

M1 = max (15,12) = 12

M2 = max(8, 20) = 20

M3 = max(-14,15) = 15

Chance node trái: C1 = 0.2*15+0.5*20+0.3*15 = 17.5

Nhánh giữa:

M4=-2

M5=-18

M6=15

Chance node giữa: C2 = 3.2

Nhánh trái:

M7 = 18

M8 = 20

M9 = -10

Chance node: C3 = 7.6

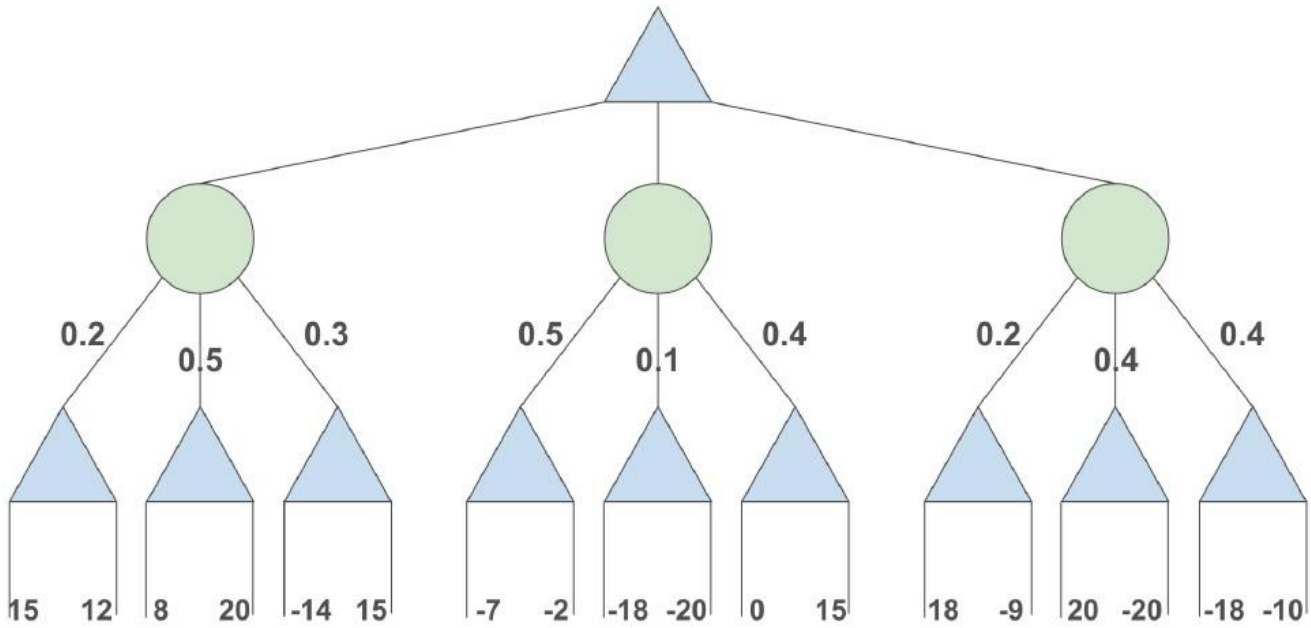
Nút gốc: V(A) = max(17.5, 3.2, 7.6) = 17.5 => Chọn nhánh trái

b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Không thể cắt tỉa an toàn các nhánh chưa biết giá trị trong cây này.

Vì chance node tính kỳ vọng dựa trên tất cả các kết quả có xác suất khác 0, một nhánh chưa xét có thể mang giá trị rất lớn và làm thay đổi kết quả. Khi chưa biết giới hạn giá trị lá, ta không thể xác định một cận trên đủ chặt để loại bỏ nhánh đó.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Ta có thể sử dụng giới hạn trên để xác định một chance node có khả năng vượt qua nhánh tốt nhất hiện tại hay không.

Xét nhánh trái: C1=17.5

⇨ Giá trị tốt nhất tại gốc hiện tại: $\alpha = 17.5$

Xét nhánh giữa:

M4 = max(-7,-2) = -2

Giả sử 2 nút max chưa xét đều đạt GTLN có thể là 20

Cận trên node giữa: UB(C2) = 0.5*(-2)+0.1*(20)+0.4*20 = 9

Vì 9 < 17.5 => Nhánh giữa không thể tốt hơn nhánh trái.

⇨ Cắt tỉa 2 nhánh max còn lại của chance node giữa , gồm các lá (-18,-20) và (0,15)

Xét nhánh phải:

M7 = 18

Cận trên kia chưa xét 2 max còn lại: UB(C3)= 0.2*18+0.4*20 +0.4*20 = 19.6

Vì 19.6>17.5, chưa thể cắt toàn bộ phần còn lại

Xét max thứ 2: Gặp lá 20 => đạt GTLN => không cần xét lá -20

Max cuối: M9 = max(-18, -10) = -10

C3 = 0.2*18+0.4*20+0.4*(-10) = 7.6

⇨ V(A)=17.5

Các nhánh có thể cắt:

- 2 nút max còn lại dưới chance node giữa : (-18, -20) và (0, 15)
- Lá -20 thuộc MAX thứ 2 của nhánh phải.

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Tiêu chí	Forward Checking	AC-3
Phạm vi	Chỉ kiểm tra các biến chưa gán có ràng buộc trực tiếp với biến vừa gán	Kiểm tra và lan truyền giữa các biến trên các cung ràng buộc
Thời điểm	Sau mỗi phép gán trong quá trình tìm kiếm	Có thể trước hoặc trong quá trình tìm kiếm
Lan truyền	Không tiếp tục lan truyền giữa các biến chưa gán	Tiếp tục lan truyền khi miền thay đổi
Chi phí	Thường thấp hơn	Thường cao hơn
Hiệu quả	Phát hiện một số xung đột sớm	Có thể phát hiện nhiều xung đột sớm hơn

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV)

MRV ưu tiên chọn biến có ít giá trị hợp lệ còn lại nhất. Nhờ đó, những biến khó được xử lý trước, giúp phát hiện xung đột sớm và giảm số lần quay lui (backtracking).

ii) Least Constraining Value (LCV)

LCV ưu tiên chọn giá trị loại bỏ ít lựa chọn nhất của các biến chưa gán. Nhờ vậy, các biến còn lại có nhiều khả năng tìm được nghiệm, giúp hạn chế backtracking.

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

- Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.
- Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.
- Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.
- Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C.
- Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng.

a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

Biến: X = {A, B, C, D, E}

Mỗi biến biểu diễn lịch của một máy bay, gồm: Xi = (ti, ri)

Trong đó:

ti = Khung giờ.

ri = Đường băng.

Miền giá trị: Di={1,2,3,4} × {QT, ND}

(QT : quốc tế, ND: nội địa)

Ràng buộc unary:

tB = 1,

tD >=3

tA <= 2

Ràng buộc binary:

Máy bay D hạ cánh trước khi C cất cánh: tD < tC

Không có hai máy bay cùng khung giờ trên cùng đường băng:

$$\forall i \neq j, (ti \neq tj) \vee (ri \neq rj)$$

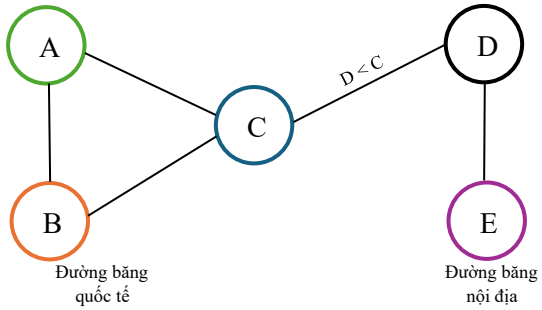
Bổ sung hai ràng buộc sau:

- A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.
- D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

Miền ban đầu: DA = DB = DC = DD= DE = {1,2,3,4}

Đồ thị ràng buộc:



c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

Biến	Miền ban đầu	Miền sau unary
A	1234	12
B	1234	1
C	1234	1234
D	1234	34
E	1234	1234

*Áp dụng AC-3:

Vì B = 1 => A và C ≠ 1

D≥ 3 và D < C => C = 4

C = 4 => D = 3

A ≠ C, D => A = 2

D = 3 và E chung đường băng với D => E ≠ 3

Miền cuối cùng:

Biến	Miền sau AC-3
A	2
B	1
C	4
D	3
E	124

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

Chọn B = 1 vì B chỉ có 1 giá trị.

Forward Checking : loại A = 1 => Chọn A = 2

Lúc này C và D đều có 2 giá trị {3,4}. Ưu tiên C => Chọn C = 3

D phải nhỏ hơn C, mà D ∈ {3,4} => Không có giá trị phù hợp cho D => Dead end.

Backtrack: Đổi C = 4 => Chọn D = 3.

Chọn E = 1 (giá trị nhỏ nhất còn hợp lệ)

*Thứ tự phép gán:

(B,1) → (A,2) → (C,3)

Backtrack → (C,4) → (D,3) → (E,1)

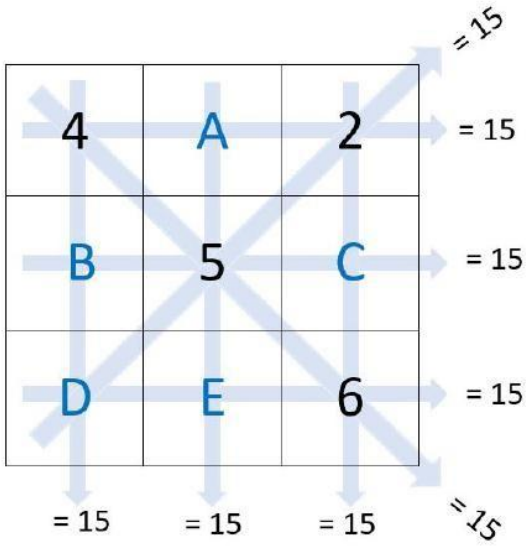
*Nghiệm cuối cùng:

Máy bay	**Khung giờ**	**Đường băng**
A	2	Quốc tế
B	1	Quốc tế
C	4	Quốc tế
D	3	Nội địa
E	1	Nội địa

Máy bay B và E có thể cùng khung giờ 1 vì sử dụng hai đường băng khác nhau.

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n^2}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

*Biến: X = {A, B, C, D, E}

*Miền giá trị:

Các số đã biết: 2, 4, 5, 6.

⇨ Các số chưa sử dụng: {1, 3, 7, 8, 9}

*Các ràng buộc:

Mỗi hàng có tổng = 15:

4 + A + 2 = 15

B + 5 + C = 15

D + E + 6 = 15

Mỗi cột có tổng = 15:

4 + B + D = 15

A + 5 + E = 15

2 + C + 6 =15

Hai đường chéo có tổng = 15

4 + 5 + 6 =15

2 + 5 + D = 15

Tất cả các ô phải chứa giá trị khác nhau: AllDiff(A, B, C, D, E)

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.

Bỏ qua ràng buộc alldiff, các quan hệ giữa các biến là:

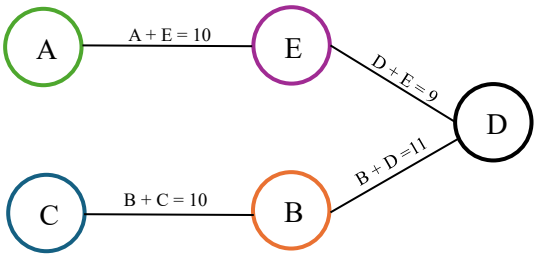
B + C = 10

D + E = 9

B + D = 11

A + E = 10

Các ràng buộc còn lại trở thành unary, xác định A, C và D



c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

4 + A + 2 = 15 => A = 9

2 + C + 6 = 15 => C = 7

2 + 5 + D = 15 => D = 8

Vì A + E = 10 => 9 + E = 10 => E = 1

Vì B + C = 10 => B + 7 = 10 => B = 3

Kết quả:

4	9	2
3	5	7
8	1	6

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

- AJ = {Đỏ, Vàng, Xanh lá} => A
- Sunny = {Đỏ, Vàng, Hồng} => S
- Ellyse = {Đỏ, Hồng} => E

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

Chọn Ellyse vì có miền nhỏ nhất.

*Chọn màu bằng LCV:

E có :

- Đỏ: Loại màu Đỏ khỏi cả A và S (2 lựa chọn)
- Hồng: Chỉ loại màu Hồng khỏi S (1 lựa chọn)

LCV chọn Hồng : E = Hồng

A còn 3 màu.

S còn 2 màu.

⇨ Chọn S (MRV).

S còn 2 màu ràng buộc như đối với A, theo thứ tự cầu vòng thì chọn Đỏ.

⇨ S = Đỏ

A còn => Chọn Vàng (theo thứ tự cầu vòng)

⇨ A = Vàng

Kết quả:

Thứ tự	Sinh viên	Màu được chọn
1	Ellyse	Hồng
2	Sunny	Đỏ
3	AJ	Vàng

⇨ Số lần backtrack = 0

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

A và S có cùng 3 giá trị => Chọn A (theo thứ tự xuất hiện trong đề)

Với A:

- Đỏ: Loại 2 lựa chọn của S và E
- Vàng: Loại 1 ở S
- Xanh lá: không loại lựa chọn nào.

Chọn: A = Đỏ. (chọn màu loại nhiều lựa chọn nhất.)

S còn 2 màu.

E còn 1 màu.

⇨ Chọn S.

S có:

- Vàng: E vẫn còn hồng → Không loại giá trị nào.
- Hồng: Loại 1 giá trị ở E.

⇨ Chọn Hồng.

⇨ S = Hồng.

*Dead end:

Khi S chọn Hồng: E = {∅} .

E không còn màu hợp lệ => Quay lui về phép gán cho S = Hồng để thử màu khác.

Backtracking:

Thay: S = Vàng

Lúc này:

A = Đỏ

S = Vàng

E vẫn còn Hồng.

Gán màu cho E: E = Hồng

⇨ Tất cả các học sinh đều có màu khác nhau → Thõa mãn ràng buộc.

*Thứ tự gán:

Lần	Biến	Giá trị	Kết quả
1	AJ	Đỏ	Hợp lệ
2	Sunny	Hồng	Dead End
3	Sunny	Vàng	Hợp lệ sau quay lui
4	Ellyse	Hồng	Hợp lệ

Số lần backtrack = 1.

So sánh với câu (a):

Phương pháp	Backtrack
MRV + LCV	0 lần
Maximum Remaining Values + Most Constraining Value	1 lần

MRV kết hợp MCV phải quay lui 1 lần vì ưu tiên biến có nhiều lựa chọn và màu gây ràng buộc mạnh nhất. Điều này khiến miền của Ellyse trở thành rỗng, dẫn đến dead end. Trong khi đó, MRV kết hợp LCV tìm được nghiệm ngay mà không cần quay lui.

--- Hết ---